Project Document

Name : N.Lahari
Roll No: 23481A05H2
Section: CSE-C

Title : Doctor Availability & Queue Display System

Abstract

The Doctor Availability & Queue Display System is designed to provide real-time information about doctors' availability and patient queue status in hospitals and clinics. In many healthcare facilities, patients face long waiting times due to lack of proper scheduling and transparency. This system helps patients know whether a doctor is available and their position in the queue before consultation. By digitizing doctor schedules and patient queues, the system improves time management, reduces overcrowding, and enhances overall patient satisfaction. It can be implemented using display screens, web applications, or mobile interfaces.

Existing System

In the existing system, most hospitals and clinics manage doctor availability and patient queues manually. Patients usually wait in long lines without knowing the exact waiting time or queue position. Receptionists handle registrations and verbally inform patients about doctor availability, which often leads to confusion and errors. There is no centralized or real-time update system, resulting in inefficient time utilization for both doctors and patients.

Problems in Existing System

- No real-time doctor availability information
- Manual queue management
- Long waiting times for patients
- Lack of transparency
- Higher chances of human error

Proposed System

The proposed Doctor Availability & Queue Display System automates the process of displaying doctor availability and patient queue status. Doctors can update their availability (Available, Busy, On Break, Not Available) through the system. Patients are assigned a token number during registration, and the current queue number is displayed on digital screens or applications. The system continuously updates the queue and notifies patients when their turn is approaching.

Advantages of Proposed System

- Real-time doctor availability updates
- Automated queue management
- Reduced waiting time
- Improved patient experience
- Efficient use of hospital resources

Architecture

The system architecture consists of the following components:

1. **User Interface (UI)**
    - Display screens, web app, or mobile app
    - Shows doctor availability and queue status
2. **Registration Module**
    - Generates patient token numbers

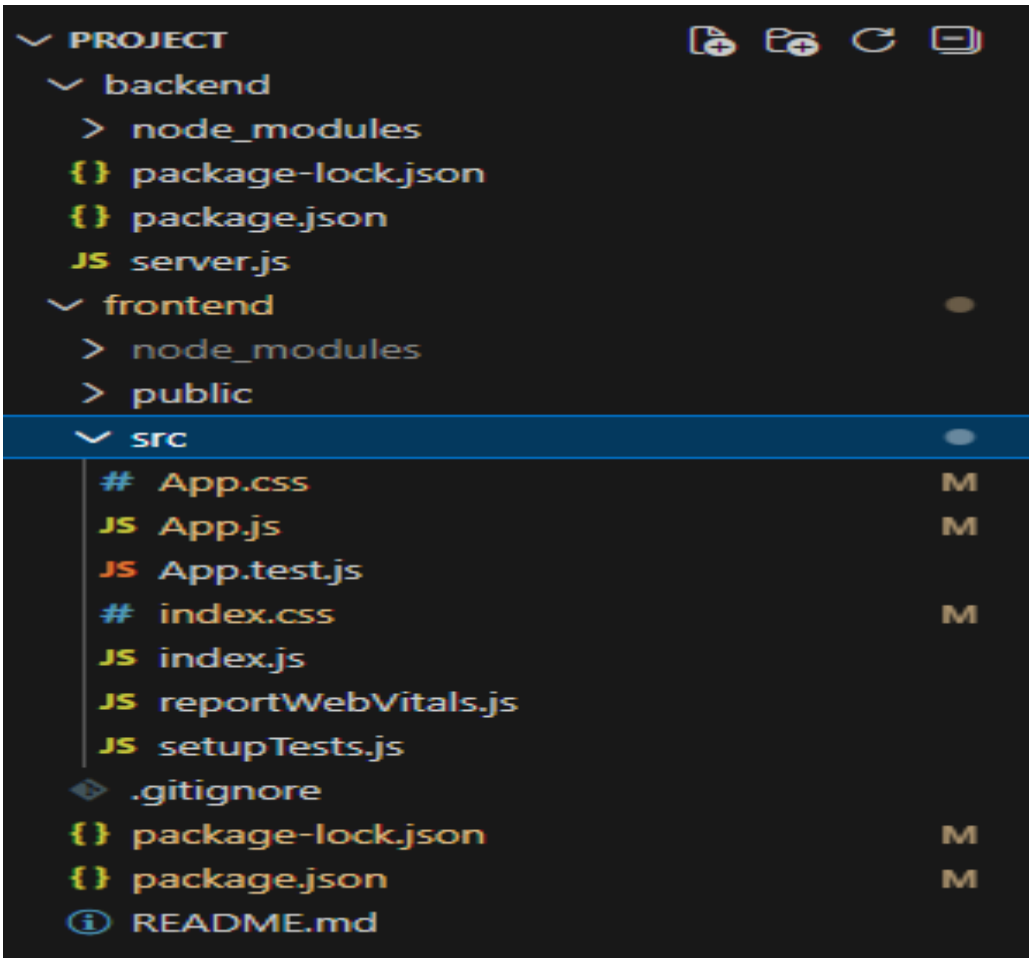
    - Stores patient data
3. **Doctor Module**
    - Allows doctors to update their availability
4. **Database**
    - Stores doctor schedules, patient tokens, and queue details
5. **Server / Backend**
    - Processes requests
    - Updates queue and availability in real time

Architecture Flow

- Patient registers → Token generated
- Doctor updates availability
- Server updates database
- Display unit shows live queue and doctor status

<u>Source code</u>

<u>Project structure:</u>



Server.js

```
const express = require('express');
```

```
const app = express();

const http = require('http').createServer(app);

const io = require('socket.io')(http, { cors: { origin: "*" } });

let queue = [];

let docStatuses = {};

let serving = {};

io.on('connection', (socket) => {

    socket.emit('update_all', { queue, docStatuses, serving });



    socket.on('join_queue', (data) => {

        const token = "TKT-" + Math.floor(100 + Math.random() * 900);

        queue.push({ ...data, token });

        io.emit('update_all', { queue, docStatuses, serving });

    });

    socket.on('next_patient', (doctorName) => {

        const index = queue.findIndex(p => p.doctorName === doctorName);

        if (index !== -1) {

            serving[doctorName] = queue.splice(index, 1)[0];

            io.emit('update_all', { queue, docStatuses, serving });

        }

    });

    socket.on('end_consultation', (doctorName) => {

        serving[doctorName] = null;

        io.emit('update_all', { queue, docStatuses, serving });

    });

    socket.on('update_doctor_status', (data) => {

        docStatuses[data.doctorId] = data.status;

        io.emit('update_all', { queue, docStatuses, serving });

    });
```

```
});

http.listen(3001, () => console.log("Server running on port 3001"));
```

App.js

```
import React, { useState, useEffect } from 'react';

import { BrowserRouter as Router, Routes, Route, useNavigate, useParams, Link } from 'react-router-dom';

import io from 'socket.io-client';

import './App.css';

const socket = io("http://localhost:3001");

const DOCTORS_LIST = [

  { id: 1, name: "Dr. Sharma", dept: "Cardiology", room: "201", pass: "pass1" },

  { id: 2, name: "Dr. Mehta", dept: "Dermatology", room: "202", pass: "pass2" },

  { id: 3, name: "Dr. Kumar", dept: "Pediatrics", room: "203", pass: "pass3" },

  { id: 4, name: "Dr. Verma", dept: "Orthopedics", room: "204", pass: "pass4" },

  { id: 5, name: "Dr. Singh", dept: "Neurology", room: "205", pass: "pass5" }

];

function App() {

  const [queue, setQueue] = useState([]);

  const [docStatuses, setDocStatuses] = useState({});

  const [serving, setServing] = useState({});

  useEffect(() => {

    socket.on('update_all', (data) => {

      setQueue(data.queue || []);

      setDocStatuses(data.docStatuses || {});

      setServing(data.serving || {});

    });

  }, []);
```

```
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/receptionist" element={<Receptionist queue={queue} />} />
        <Route path="/doctor-login" element={<DoctorLogin />} />
        <Route path="/doctor/:id" element={<DoctorDashboard docStatuses={docStatuses} queue={queue} serving={serving} />} />
        <Route path="/public" element={<PublicDisplay docStatuses={docStatuses} queue={queue} serving={serving} />} />
      </Routes>
    </Router>
  );
}
// --- RECEPTIONIST & DOCTOR DASHBOARDS ---
const Receptionist = ({ queue }) => {
    const [name, setName] = useState("");
    const [selectedDoc, setSelectedDoc] = useState(DOCTORS_LIST[0].name);
    const navigate = useNavigate();
    const handleRegister = () => {
      if (name.trim()) {
        socket.emit('join_queue', { name, doctorName: selectedDoc });
        setName("");
      }
    };
    return (
      <div className="admin-layout">
        <header className="admin-nav">
          <h2>Receptionist Dashboard</h2>
          <button className="btn-logout-small" onClick={() => navigate('/')}>Logout</button>
```

```
      </header>
      <div className="admin-content">
        <div className="registration-card-box">
          <h3>Register Patient</h3>
          <div className="reg-form">
            <input value={name} onChange={e => setName(e.target.value)} placeholder="Full Patient Name" />
            <select value={selectedDoc} onChange={e => setSelectedDoc(e.target.value)}>
              {DOCTORS_LIST.map(d => <option key={d.id} value={d.name}>{d.name} ({d.dept})</option>)}
            </select>
            <button className="btn-submit" onClick={handleRegister}>Add to Queue</button>
          </div>
        </div>
        <div className="queue-table-card">
          <h3>Current Hospital Queue</h3>
          <table className="pro-table">
            <thead>
              <tr><th>Token</th><th>Patient Name</th><th>Doctor Assigned</th></tr>
            </thead>
            <tbody>
              {queue.map((p, i) => (
                <tr key={i}><td><strong>{p.token}</strong></td><td>{p.name}</td><td>{p.doctorName}</td></tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  );
};
```

```
<div className="card serving-section">
  <p style={{fontWeight:'bold', color:'#64748b'}}>NOW SERVING</p>
  {currentServing ? (
    <div>
      <h1 style={{fontSize:'2.5rem'}}>{currentServing.name}</h1>
      <div className="token-badge">{currentServing.token}</div>
      <button className="btn-end" onClick={() => socket.emit('end_consultation', doc.name)}>End Consultation</button>
    </div>
  ) : (
    <div>
      <p style={{margin:'20px 0'}}>Room is currently empty</p>
      <button className="btn-call" disabled={myQueue.length === 0} onClick={() => socket.emit('next_patient', doc.name)}>Call Next Patient</button>
    </div>
  )}
</div>

<div className="card">
  <h3>Waiting ({myQueue.length})</h3>
  <div style={{marginTop:'15px'}}>
    {myQueue.map((p, i) => (
      <div key={i} style={{display:'flex', justifyContent:'space-between', padding:'10px 0', borderBottom:'1px solid #f1f5f9'}}>
        <span>{p.name}</span><strong>{p.token}</strong>
      </div>
    ))}
  </div>
</div>

<button onClick={() => navigate('/')} style={{background:'none', border:'none', color:'#64748b', cursor:'pointer', marginTop:'10px'}}>Log Out from System</button>
```

```
      </div>
    </div>
  );
};
const PublicDisplay = ({ docStatuses, queue, serving }) => {
  const [time, setTime] = useState(new Date().toLocaleTimeString());
  useEffect(() => {
    const timer = setInterval(() => setTime(new Date().toLocaleTimeString()), 1000);
    return () => clearInterval(timer);
  }, []);
  return (
    <div className="tv-layout">
      <header className="tv-header">
        <h1>HOSPITAL QUEUE STATUS</h1>
        <div className="tv-clock">{time}</div>
      </header>
      <main className="tv-body">
        <div className="tv-card">
          <h3 style={{marginBottom:'20px', color:'#94a3b8'}}>DOCTOR STATUS</h3>
          {DOCTORS_LIST.map(d => (
            <div key={d.id} className="tv-row">
              <div><strong>{d.name}</strong><br/><small>Room {d.room}</small></div>
              <span className={`status-tag ${docStatuses[d.id] || 'Available'}`}>{docStatuses[d.id] || 'Available'}</span>
            </div>
          ))}
        </div>
      <footer className="tv-footer">
        <span style={{fontSize:'1.5rem', fontWeight:'900'}}>NOW SERVING:</span>
```

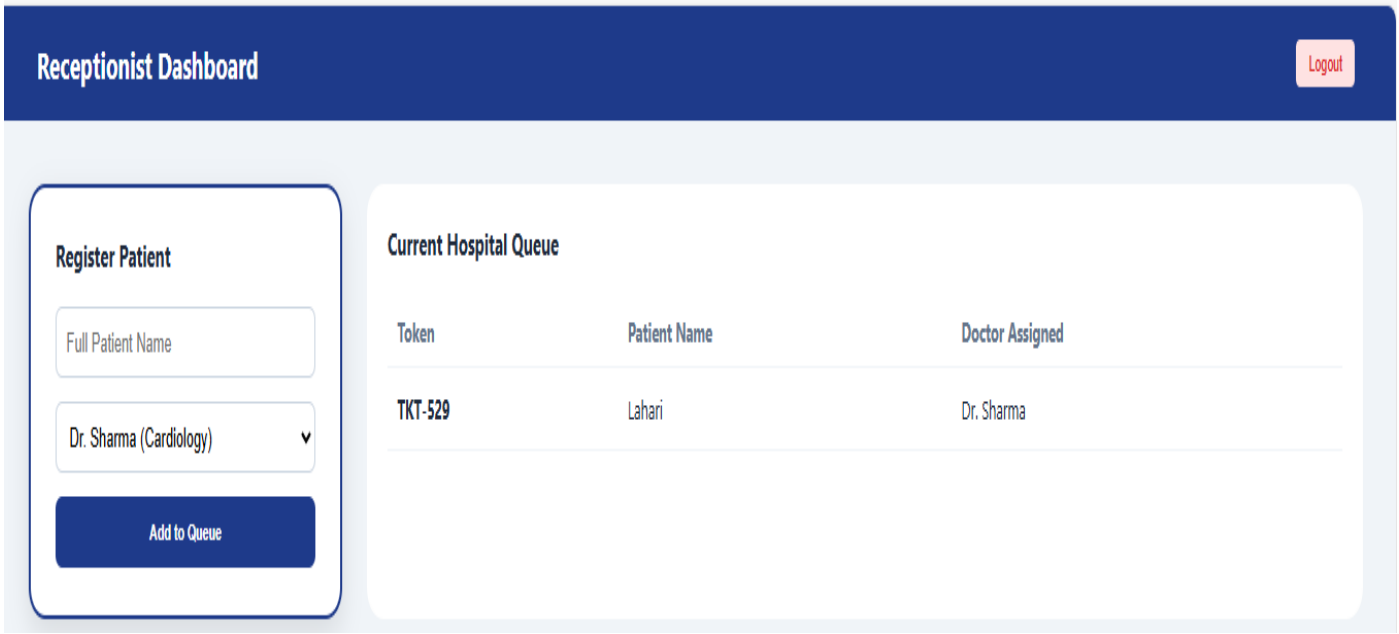
```
        {Object.entries(serving).map(([doc, p]) => p && (
          <div key={doc} className="serving-item">
            <small>Room {DOCTORS_LIST.find(d => d.name === doc)?.room}</small>
            <strong>{p.token}</strong>
          </div>
        ))}
      </footer>
    </div>
  );
};

export default App;
```

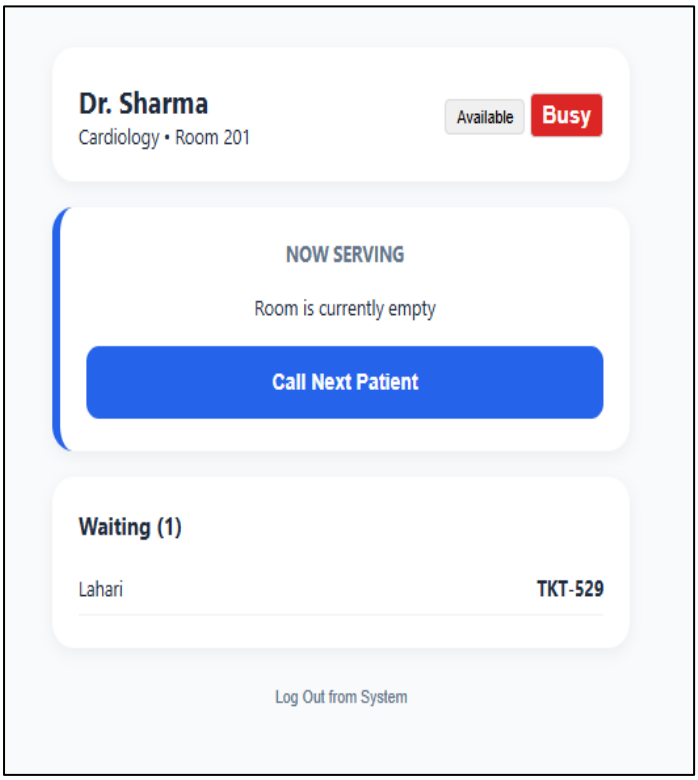
Screenshots with results

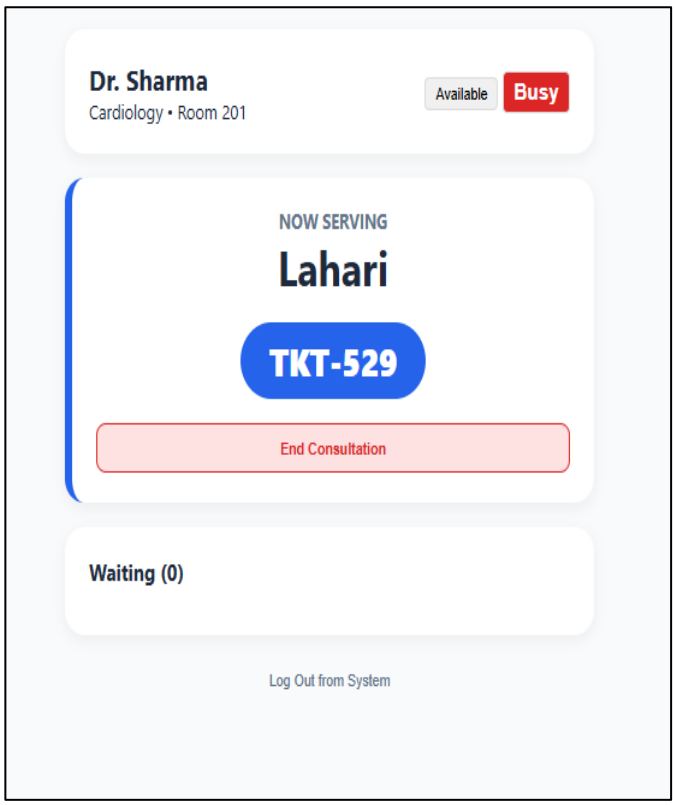
Home page:

Receptionist Dashboard:

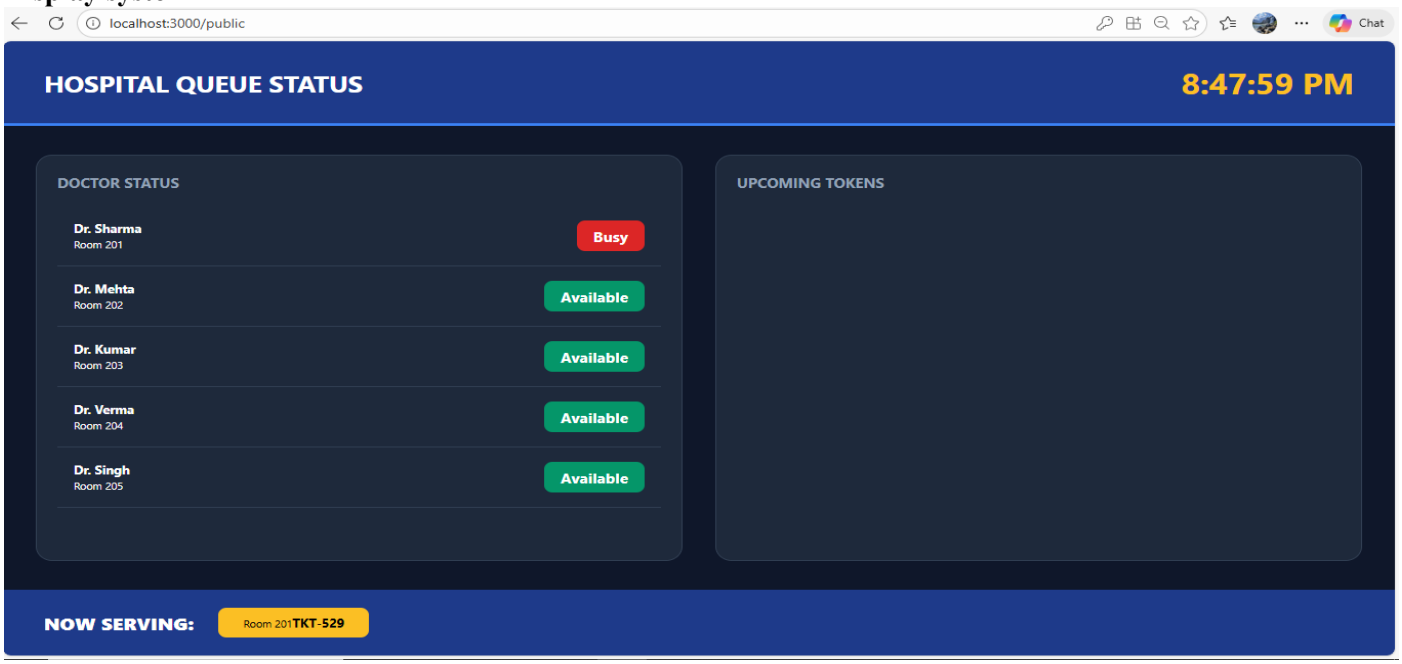


Doctor dashboard:





Display system:



Conclusion

The Doctor Availability & Queue Display System provides an efficient and transparent solution to manage patient queues and doctor schedules. By replacing manual processes with an automated system, hospitals can significantly reduce waiting times and improve service quality. This system enhances communication between patients and healthcare providers and contributes to better healthcare management. With further enhancements, it can be integrated with appointment booking and notification systems for even better performance.